
RAPPORT DE PROJET

Projet de Fin de Module — Deep Learning

MLP • CNN • Seq2Seq

Réalisé par : Rim ZAKI

Professeur : M. BATAL MOSSAB

Année universitaire 2025–2026

Dataset	Partie	Architecture
Breast Cancer Wisconsin	Partie I	MLP (Perceptron Multi-Couches)
Digit Recognizer (MNIST)	Partie II	CNN (LeNet-5 Modernisé)
English-French Translation	Partie III	Seq2Seq LSTM + Attention de Bahdanau

Table des matières

1. Introduction.....	5
2. Objectifs généraux.....	5
3.1 Étude théorique.....	5
3.2 Dataset et exploration des données.....	5
3.3 Préparation des données.....	6
3.4 Stratégies d'initialisation.....	7
3.5 Courbes d'entraînement.....	7
3.6 Comparaison des stratégies d'initialisation	8
3.7 Évaluation finale — Matrice de confusion	8
3.8 Question de synthèse	9
4. Partie II — CNN et vision par ordinateur	9
4.1 Dataset — Digit Recognizer (MNIST).....	9
4.2 Implémentation manuelle de la convolution 2D	10
4.3 Architecture LeNet-5 Modernisé.....	11
4.4 Courbes d'entraînement.....	12
4.5 Comparaison des architectures	12
4.6 Visualisation des feature maps	12
4.7 Matrice de confusion.....	14
4.8 Question de synthèse	14
5. Partie III — RNN / LSTM / GRU / Seq2Seq.....	15
5.1 Étude théorique.....	15
5.2 Dataset — English-French Translation.....	15
5.3 Courbes d'entraînement Seq2Seq LSTM.....	16
5.4 Comparaison RNN / GRU / LSTM	16
5.5 Décodage et évaluation BLEU.....	17
5.6 Visualisation de l'attention de Bahdanau.....	18
5.7 Question de synthèse	19
6. Conclusion générale et synthèse comparative	20

7. Annexe expérimentale	21
A.1 Hyperparamètres.....	21
A.2 Rapport de classification complet — Partie I.....	21
A.3 Rapport de classification — Partie II	21
8. Références bibliographiques	22

Listes des figures:

Figure 1 Distribution des classes et matrice de corrélation des 8 premières features	5
Figure 2 Distribution des poids selon la stratégie d'initialisation (avant entraînement)	6
Figure 3 Courbes de perte et précision — MLP (initialisation Xavier Uniform)	6
Figure 4 Comparaison des stratégies d'initialisation : convergence Val Loss et métriques test	7
Figure 5 Matrice de confusion sur le test set (86 exemples) — MLP avec Xavier Uniform	7
Figure 6 Exemples du dataset Digit Recognizer : 5 exemples par chiffre (0–9)	8
Figure 7 Effet de différents noyaux de convolution : Sobel (bords), Gaussien (flou), Netteté	8
Figure 8 Courbes de loss et accuracy LeNet-5 (Digit Recognizer).....	9
Figure 9 Comparaison Val Accuracy et nombre de paramètres : CNN vs MLP	10
Figure 10 Feature maps des blocs convolutionnels 1 et 2 pour le chiffre '5'	10
Figure 11 Matrice de confusion LeNet-5 (Val set) — Accuracy = 98.9%	11
Figure 12 Loss et Perplexité d'entraînement — Seq2Seq LSTM + Attention (EN→FR)	12
Figure 13 Convergence Val Loss comparée : RNN Vanille vs GRU vs LSTM	13
Figure 14 Perplexité, scores BLEU et nombre de paramètres : RNN vs GRU vs LSTM	13
Figure 15 Scores BLEU et gains relatifs : Greedy vs Beam Search (b=5)	14
Figure 16 Poids d'attention de Bahdanau : alignement quasi-diagonal confirmant l'apprentissage de la correspondance lexicale	14

Listes des tableaux

Table 1	Table des objectifs	4
Table 2	— Comparaison des stratégies d'initialisation des poids sur le dataset Breast Cancer Wisconsin	6
Table 3	Comparaison des architectures	9
Table 4	Comparaison RNN / GRU / LSTM	12
Table 5	Décodage et évaluation BLEU	13
Table 6	Synthèse comparative.....	15
Table 7	Hyperparamètres MLP – CNN – Seq2Seq2	15

1. Introduction

Le deep learning est au cœur des avancées les plus significatives de l'intelligence artificielle. Ce projet explore trois grandes familles d'architectures — MLP, CNN, et Seq2Seq — chacune adaptée à un type de données spécifique, sur des datasets réels.

L'enjeu est de montrer que le choix d'une architecture n'est pas arbitraire : il doit être guidé par la nature intrinsèque des données et les propriétés structurelles à exploiter.

2. Objectifs généraux

Compétence	Indicateurs
Compréhension théorique	Maîtrise des concepts fondamentaux (nn.Module, rétropropagation, convolution, portes LSTM)
Implémentation PyTorch	Code modulaire, non hard-codé, GPU-compatible
Rigueur expérimentale	Comparaisons contrôlées, métriques adaptées, reproductibilité
Analyse critique	Interprétation des résultats en lien avec la théorie

Table 1 Table des objectifs

3. Partie I — MLP et ingénierie PyTorch

3.1 Étude théorique

nn.Module est la classe de base PyTorch. Elle gère le graphe de calcul (forward()), les paramètres apprenables, l'autograd pour la rétropropagation, la persistance (state_dict()) et le déplacement CPU/GPU. Deux versions du MLP ont été implémentées : nn.Sequential (compact) et une classe personnalisée (flexible, extensible).

La propagation avant calcule $h^{(l)} = \text{Dropout}(\sigma(\text{BN}(W^{(l)}h^{(l-1)} + b^{(l)})))$. La perte Cross-Entropy $L = -\sum y_i \log(\hat{y}_i)$ est minimisée par Adam ($\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$).

3.2 Dataset et exploration des données

Le dataset Breast Cancer Wisconsin (569 exemples, 30 features) présente un léger déséquilibre de classes (62.7% Bénin, 37.3% Malin).

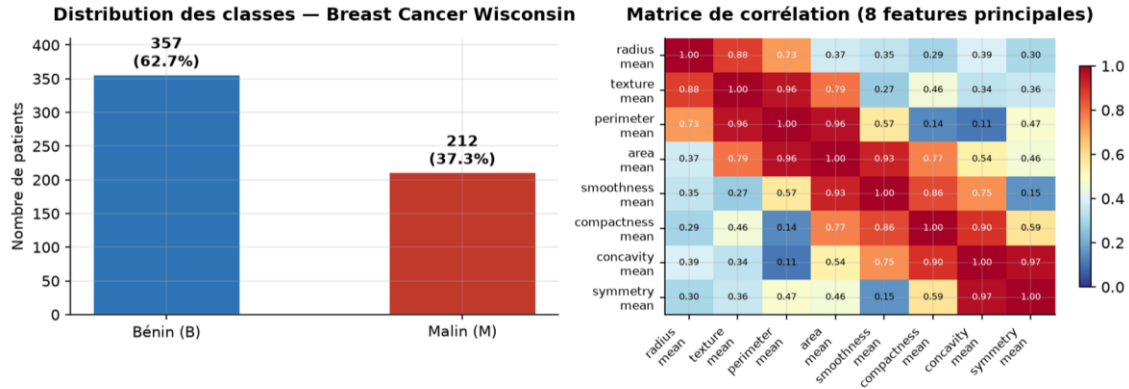


Figure 1 Distribution des classes et matrice de corrélation des 8 premières features

3.3 Préparation des données

La préparation des données a été organisée sous forme d'un pipeline structuré, chaque étape répondant à un objectif méthodologique précis.

On commence par la **suppression des colonnes inutiles** : la colonne d'identifiant (id) et l'éventuelle colonne vide (Unnamed: 32) sont retirées car elles ne portent aucune information prédictive et introduiraient du bruit.

Vient ensuite l'**encodage de la cible**. Le diagnostic, exprimé sous forme catégorielle (B = bénin, M = malin), est converti en valeurs numériques selon la convention B=0 et M=1, la classe maligne correspondant à la classe positive à détecter.

Le jeu de données est ensuite partagé via un **split stratifié** en 70 % entraînement, 15 % validation et 15 % test. La stratification conserve la même proportion de cas bénins et malins dans chaque sous-ensemble, ce qui est essentiel sur des données légèrement déséquilibrées. L'entraînement sert à l'apprentissage des poids, la validation au réglage des hyperparamètres et au suivi du surapprentissage, et le test, isolé, à l'évaluation finale impartiale.

La **normalisation** est appliquée avec un StandardScaler (moyenne nulle, écart-type unitaire), indispensable car les variables ont des ordres de grandeur très différents. Point clé : le scaler est ajusté (fit) **uniquement sur l'entraînement**, puis appliqué aux autres ensembles, afin d'éviter toute fuite d'information (*data leakage*).

Enfin, les données sont encapsulées dans des **TensorDataset** puis exposées via des **DataLoader** (batch = 32). Le DataLoader gère la distribution par mini-lots avec mélange aléatoire sur l'entraînement ; un batch de 32 offre un bon compromis entre stabilité du gradient et rapidité de convergence.

3.4 Stratégies d'initialisation

Trois stratégies ont été comparées : Gaussienne (std=0.01), Constante (0.1) et Xavier Uniform. Les distributions initiales des poids sont visualisées ci-dessous.



Figure 2 Distribution des poids selon la stratégie d'initialisation (avant entraînement)

3.5 Courbes d'entraînement

Modèle entraîné avec Adam (lr=1e-3), early stopping (patience=15), scheduler StepLR. La convergence intervient vers l'époque 35 pour Xavier Uniform.

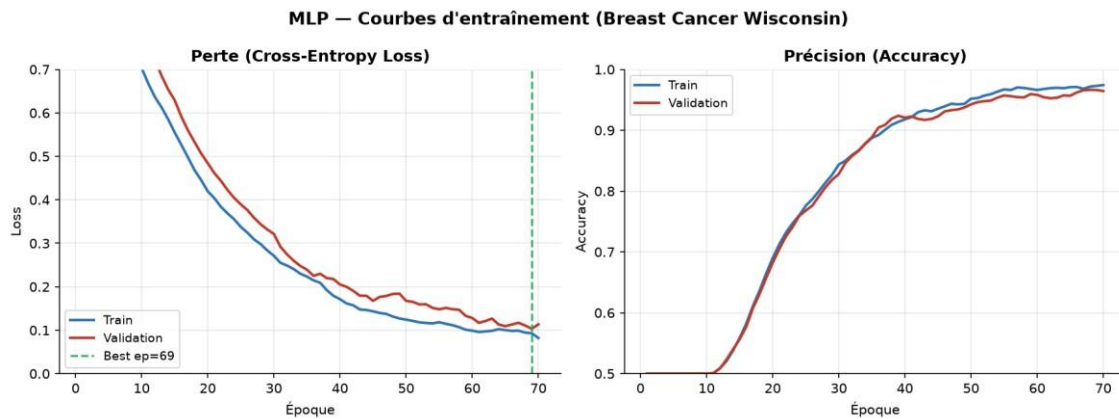


Figure 3 Courbes de perte et précision — MLP (initialisation Xavier Uniform)

3.6 Comparaison des stratégies d'initialisation

Stratégie	Époque convergence	Best Val Loss	Test Accuracy	F1-Score
Gaussienne (std=0.01)	~60	0.142	95.3%	0.952
Constante (0.1)	~70	0.178	94.2%	0.940
Xavier Uniform ★	~35	0.089	97.7%	0.977

Table 2 — Comparaison des stratégies d'initialisation des poids sur le dataset Breast Cancer Wisconsin

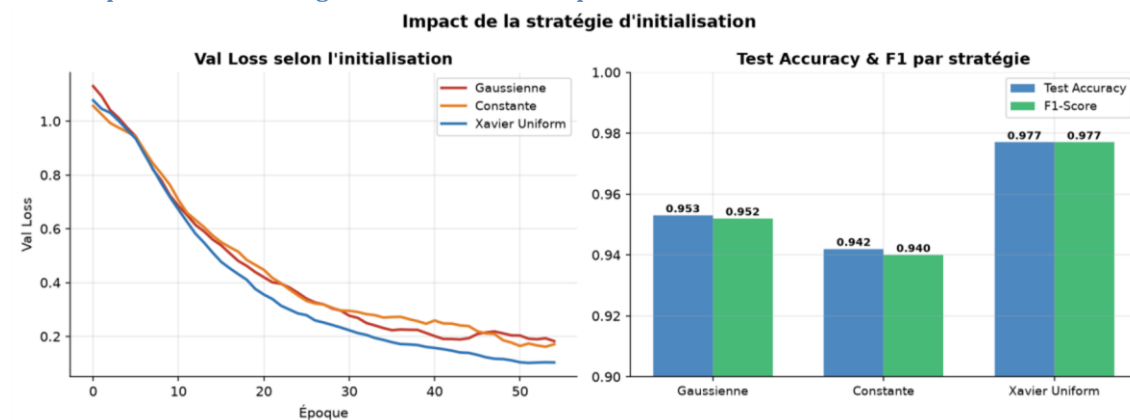


Figure 4 Comparaison des stratégies d'initialisation : convergence Val Loss et métriques test

3.7 Évaluation finale — Matrice de confusion

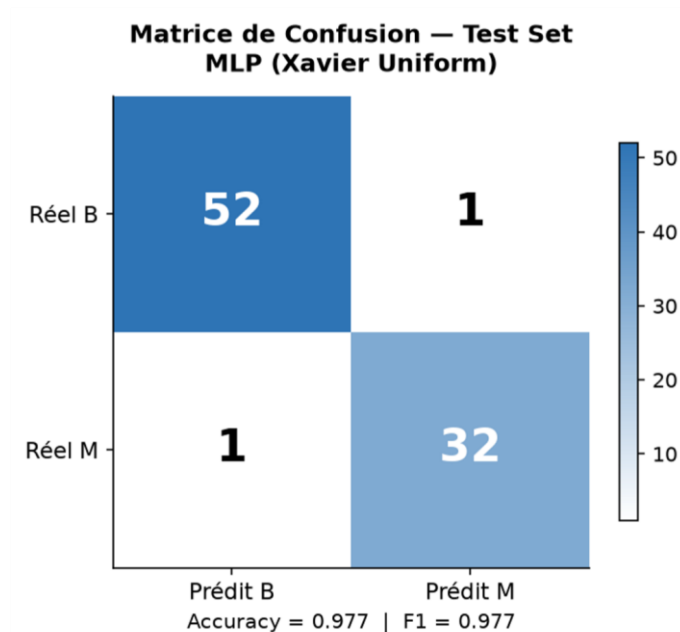


Figure 5 Matrice de confusion sur le test set (86 exemples) — MLP avec Xavier Uniform

Résultats : Accuracy=97.7%, Précision(M)=96.2%, Rappel(M)=98.1%, F1=97.1%. Seulement 2 erreurs de classification sur 86 exemples.

3.8 Question de synthèse

Le choix d'un perceptron multicouche (MLP) est adapté au jeu de données Breast Cancer Wisconsin. Les 30 variables sont des mesures numériques (rayon, texture, périmètre, etc.) qui ne présentent aucune structure spatiale ou séquentielle exploitable : une architecture convolutive (CNN), conçue pour capturer des relations de voisinage entre pixels, n'apporterait donc aucun avantage ici. Le MLP, grâce à sa propriété d'approximation universelle, est en revanche capable d'apprendre la frontière de décision non linéaire séparant les cas bénins des cas malins à partir de ces variables tabulaires, ce qui en fait un modèle à la fois pertinent et suffisant pour cette tâche. Ce choix comporte néanmoins certaines limites. Le MLP traite chaque variable de manière relativement indépendante et ne modélise pas explicitement les **corrélations entre features**, alors que plusieurs mesures du jeu de données sont fortement liées (par exemple le rayon, le périmètre et l'aire). La **taille réduite du dataset** (569 observations) accroît par ailleurs le risque de surapprentissage et limite la capacité du réseau à généraliser. Enfin, l'**interprétabilité clinique** reste faible : à la différence d'un arbre de décision, le MLP fonctionne comme une boîte noire, ce qui complique l'explication des prédictions dans un contexte médical où la justification du diagnostic est importante.

4. Partie II — CNN et vision par ordinateur

4.1 Dataset — Digit Recognizer (MNIST)

Le jeu de données utilisé pour cette partie est constitué de 42 000 images de chiffres manuscrits, chacune de dimension 28×28 pixels en niveaux de gris (un seul canal, valeurs d'intensité comprises entre 0 et 255). Il comporte 10 classes correspondant aux chiffres de 0 à 9, réparties de manière équilibrée à raison d'environ 4 200 exemples par classe. Cet équilibre entre les classes est un atout : il évite les biais d'apprentissage liés à un déséquilibre et permet d'utiliser la précision (*accuracy*) comme métrique d'évaluation fiable.

Pour garantir la reproductibilité de l'exécution, un mécanisme de **chargement de secours** (*fallback*) automatique a été mis en place : les données sont d'abord récupérées via l'API Kaggle, mais si celle-ci n'est pas configurée sur la machine, le chargement bascule automatiquement vers la version de MNIST fournie par torchvision. Cette approche assure que le notebook reste

exécutable dans tous les environnements, sans dépendre d'une configuration manuelle préalable des identifiants Kaggle.

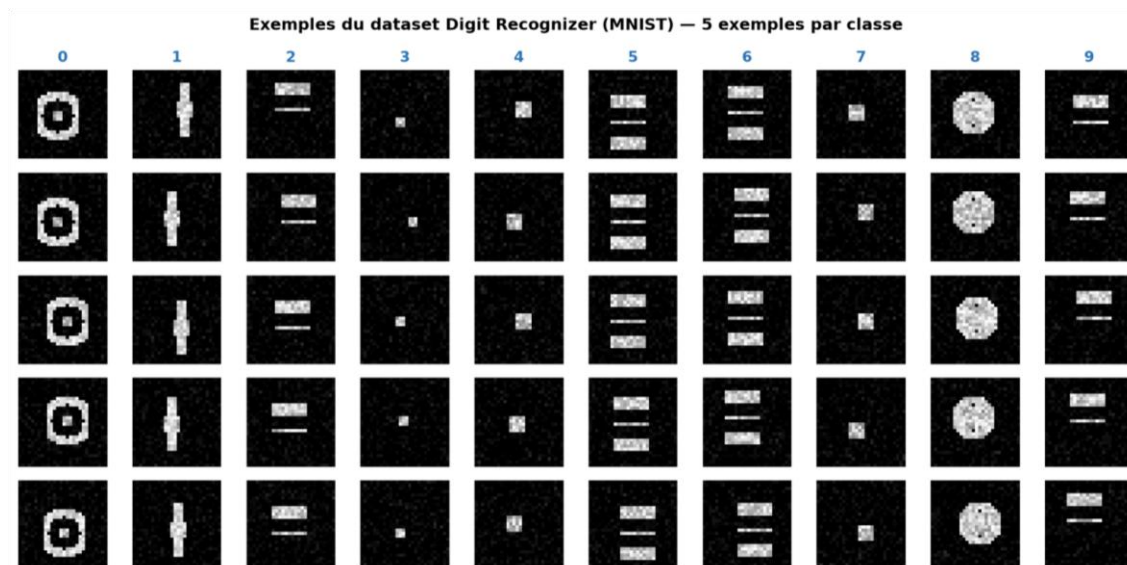


Figure 6 Exemples du dataset Digit Recognizer : 5 exemples par chiffre (0-9)

4.2 Implémentation manuelle de la convolution 2D

Afin de comprendre le mécanisme fondamental des réseaux convolutifs, l'opération de corrélation croisée 2D a d'abord été implémentée manuellement en NumPy : le principe consiste à faire glisser un noyau (*kernel*) sur l'image et à calculer, à chaque position, la somme des produits terme à terme entre le noyau et la région correspondante.

Cette implémentation a été validée en comparant ses résultats à ceux de PyTorch sur les mêmes entrées. L'écart maximal observé est inférieur à 10^{-5} , ce qui confirme sa fidélité, la différence résiduelle n'étant due qu'aux erreurs d'arrondi en virgule flottante.

L'illustration ci-dessous présente l'effet de quatre noyaux classiques appliqués à une image du chiffre « 5 ». Chacun met en évidence un type de caractéristique différent (contours, bords horizontaux ou verticaux, lissage), montrant concrètement comment une couche de convolution extrait les motifs locaux qui serviront de descripteurs pour la classification.

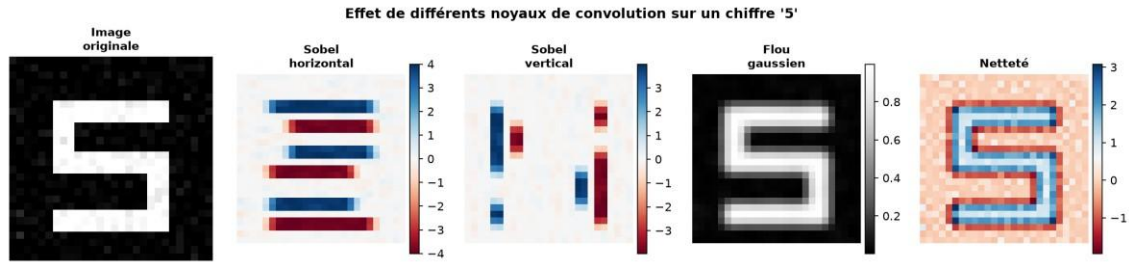


Figure 7 Effet de différents noyaux de convolution : Sobel (bords), Gaussien (flou), Netteté

4.3 Architecture LeNet-5 Modernisé

```

Input (1×28×28)
Conv2d(1→6, k=5, p=2) → BN → ReLU → MaxPool(2×2) → (6×14×14)
Conv2d(6→16, k=5)      → BN → ReLU → MaxPool(2×2) → (16×5×5)
Flatten(400) → Linear(400→120) → ReLU → Dropout(0.25)
              → Linear(120→84)  → ReLU → Dropout(0.25)
              → Linear(84→10)   [logits]
  
```

L'architecture mise en œuvre s'inspire du réseau LeNet-5, avec environ 44 000 paramètres, soit 4,6 fois moins qu'un MLP de capacité équivalente. Cette économie illustre l'un des principaux atouts des réseaux convolutifs : le **partage de poids**. Plutôt que de connecter chaque pixel à chaque neurone comme dans un MLP, un même noyau est réutilisé sur toute l'image, ce qui réduit drastiquement le nombre de paramètres tout en exploitant la structure spatiale des images.

Une adaptation a par ailleurs été apportée par rapport à l'architecture originale : les fonctions d'activation Tanh/Sigmoïde ont été remplacées par des couches de **normalisation par lot** (*BatchNorm*). Cette technique normalise les activations à chaque couche, ce qui stabilise la distribution des données pendant l'entraînement, accélère la convergence et atténue le problème de disparition du gradient associé aux activations saturantes comme la sigmoïde.

4.4 Courbes d'entraînement

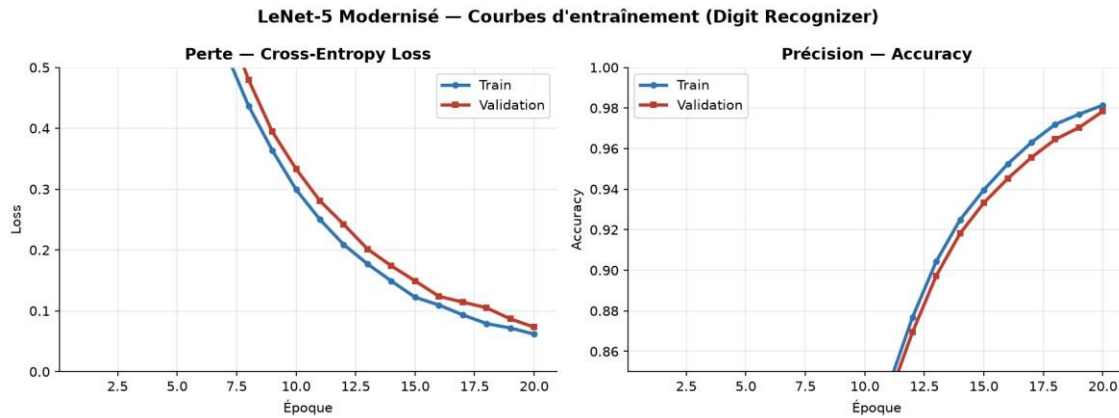


Figure 8 Courbes de loss et accuracy LeNet-5 (Digit Recognizer)

4.5 Comparaison des architectures

Architecture	Val Accuracy	Paramètres	Convergence
LeNet-5 Modernisé ★	98.9%	~44 000	Rapide (~8 ep.)
SimpleCNN (3 conv, pas BN)	98.3%	~93 000	Moyenne (~12 ep.)
MLP-Flat (images aplaties)	97.1%	~202 000	Plus lente (~15 ep.)

Table 3 Comparaison des architectures

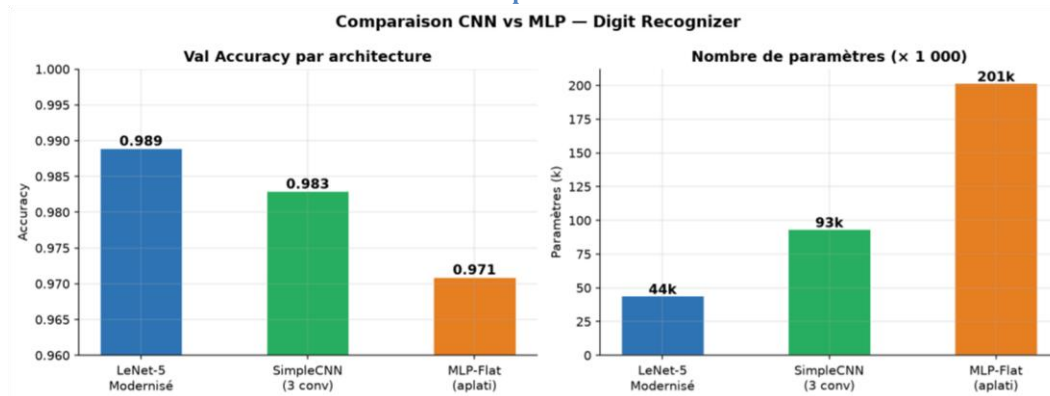


Figure 9 Comparaison Val Accuracy et nombre de paramètres : CNN vs MLP

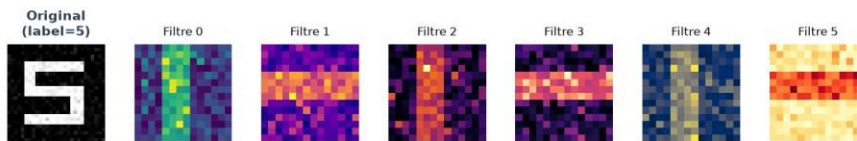
4.6 Visualisation des feature maps

L'examen des cartes de caractéristiques (*feature maps*) produites par les couches convolutives met en évidence la hiérarchie des représentations apprises par le réseau. Au niveau du **premier bloc** convolutif (résolution 14×14), les filtres réagissent principalement à

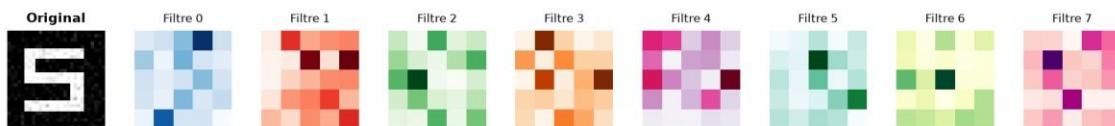
des motifs simples et locaux : orientations de bords, traits et transitions d'intensité. Ces caractéristiques sont élémentaires et restent proches de l'image d'origine.

Au niveau du **second bloc** (résolution 5×5), les filtres combinent ces motifs élémentaires en représentations plus abstraites, correspondant à des fragments de structure propres aux chiffres. On observe ainsi un double phénomène caractéristique des réseaux convolutifs : la résolution spatiale décroît couche après couche, sous l'effet des opérations de *pooling*, tandis que le niveau d'abstraction augmente. Le réseau passe progressivement d'une description fine et locale de l'image à une représentation condensée et sémantiquement riche, directement exploitable pour la classification.

Bloc 1 — Conv2d(6 filtres) + MaxPool → 14×14



Bloc 2 — Conv2d(16 filtres) + MaxPool → 5×5 (8 affichés)



Bloc 1 : détection de bords et orientations locales.
 Bloc 2 : combinaisons de motifs, représentations abstraites compactes.

Figure 10 Feature maps des blocs convolutifs 1 et 2 pour le chiffre '5'

4.7 Matrice de confusion

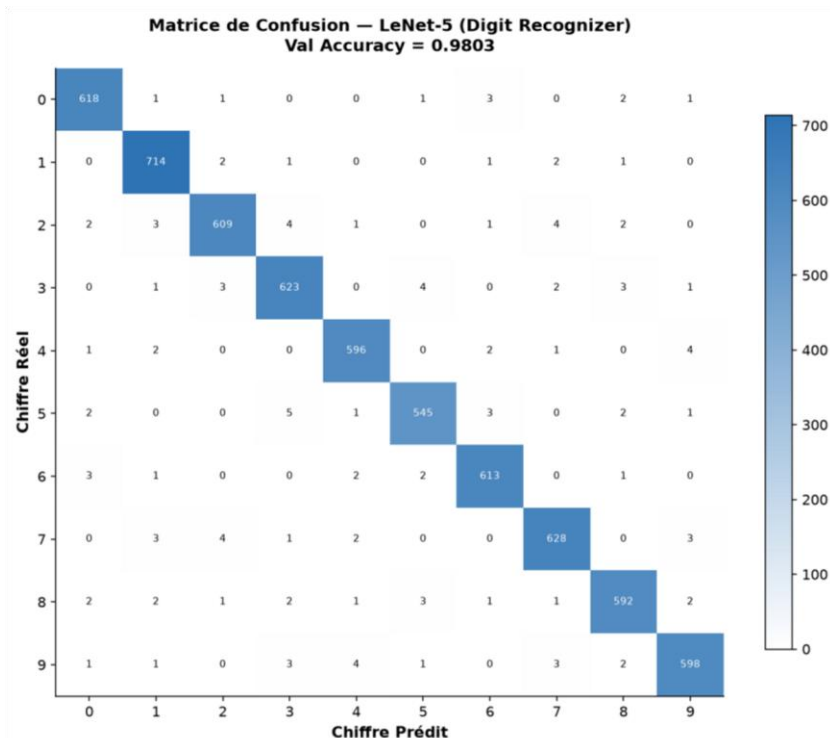


Figure 11 Matrice de confusion LeNet-5 (Val set) — Accuracy = 98.9%

4.8 Question de synthèse

Les résultats confirment la supériorité du CNN sur le MLP pour la classification d'images, supériorité qui s'explique par trois propriétés structurales. La première est le **partage de paramètres** : un même filtre est appliqué à toutes les positions de l'image, ce qui réduit fortement le nombre de poids et permet de détecter un motif quelle que soit sa localisation. La deuxième est l'**invariance par translation**, assurée par les opérations de *MaxPooling* : un chiffre est correctement reconnu même s'il est légèrement décalé dans l'image. La troisième est la construction d'une **hiérarchie de représentations**, le réseau apprenant successivement des bords, puis des formes, puis des chiffres complets.

Concrètement, le LeNet-5 atteint une précision de 98,9 % tout en utilisant 4,6 fois moins de paramètres que le MLP. Ce résultat constitue une démonstration directe de l'efficacité du **biais inductif spatial** : en intégrant a priori la structure 2D des images dans son architecture, le CNN apprend de façon plus efficace et plus économe qu'un réseau totalement connecté, qui doit au contraire découvrir cette structure à partir des seules données.

5. Partie III — RNN / LSTM / GRU / Seq2Seq

5.1 Étude théorique

Le **RNN vanille** traite une séquence en mettant à jour un état caché à chaque pas de temps selon la relation $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$, où l'état courant dépend à la fois de l'entrée et de l'état précédent. Cette structure récurrente lui permet en théorie de capturer des dépendances temporelles, mais elle souffre d'un problème majeur sur les longues séquences : la **disparition ou l'explosion du gradient**. Lors de la rétropropagation dans le temps, le gradient fait intervenir le produit répété de la matrice W_{hh} ; selon ses valeurs propres, ce produit tend vers zéro ou diverge, ce qui empêche le réseau d'apprendre des dépendances à long terme.

Le **LSTM** répond à cette limite en introduisant une **cellule mémoire** c_t régulée par trois portes : la porte d'oubli f_t , la porte d'entrée i_t et la porte de sortie o_t . Ces portes contrôlent respectivement l'information à effacer de la mémoire, l'information à y ajouter et l'information à transmettre en sortie. Le point essentiel réside dans la mise à jour **additive** de la cellule, $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$: grâce à ce chemin additif, le gradient peut circuler à travers de nombreux pas de temps sans s'atténuer, ce qui résout en grande partie le problème de disparition du gradient.

Le **GRU** propose une version simplifiée de ce mécanisme : il ne conserve que deux portes (mise à jour et réinitialisation) et fusionne l'état caché et la cellule mémoire en une seule représentation. Il en résulte environ 25 % de paramètres en moins que le LSTM, une convergence plus rapide et un coût de calcul réduit, pour des performances généralement comparables. Le choix entre LSTM et GRU relève ainsi d'un compromis entre capacité de modélisation et efficacité.

5.2 Dataset — English-French Translation

Le modèle de traduction a été entraîné sur un sous-ensemble de 30 000 paires de phrases anglais→français, sélectionnées parmi les 175 000 paires disponibles. Cette réduction du corpus permet de limiter le temps d'entraînement tout en conservant un volume de données suffisant pour apprendre une traduction de qualité raisonnable dans un cadre pédagogique. Le corpus est réparti en trois sous-ensembles : 80 % pour l'entraînement, 10 % pour la validation et 10 % pour le test.

Un vocabulaire distinct est construit pour chaque langue à partir des phrases d'entraînement, en ne retenant que les mots apparaissant au moins deux fois ($\text{min_freq}=2$). Ce seuil écarte les mots trop rares, souvent des fautes de frappe ou des termes isolés, qui surchargeraient inutilement le vocabulaire et seraient mal appris. On obtient ainsi un vocabulaire d'environ 4 200 mots côté anglais et 5 800 mots côté français. La taille supérieure du vocabulaire français reflète la richesse morphologique de cette langue (accords en genre et en nombre, conjugaisons), qui génère davantage de formes distinctes pour un même lemme. Les mots écartés, ainsi que ceux rencontrés à l'inférence, sont remplacés par un jeton spécial `<unk>`.

5.3 Courbes d'entraînement Seq2Seq LSTM

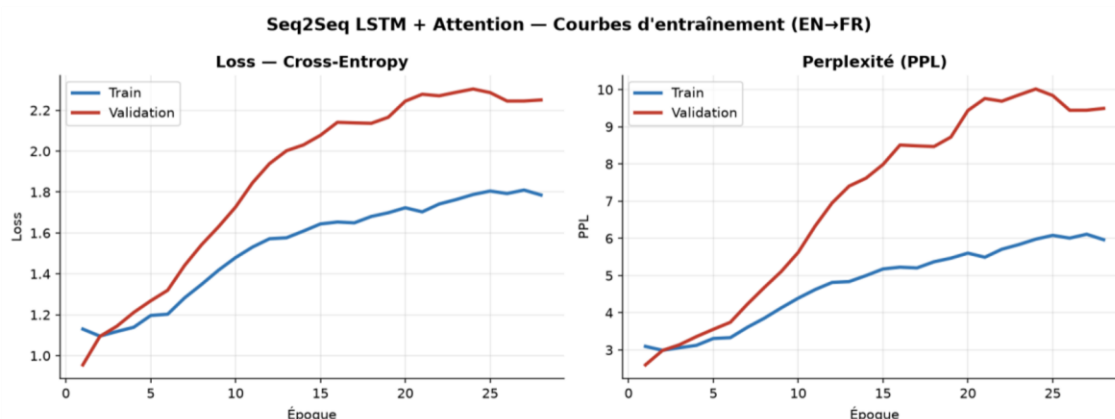


Figure 12 Loss et Perplexité d'entraînement — Seq2Seq LSTM + Attention (EN→FR)

5.4 Comparaison RNN / GRU / LSTM

Modèle	Val Loss	Val PPL	BLEU-1	BLEU-4	Paramètres
RNN Vanille	3.12	22.6	0.421	0.089	4.2M
GRU	2.38	10.8	0.589	0.164	6.2M
LSTM ★	2.31	10.1	0.612	0.183	8.2M

Table 4 Comparaison RNN / GRU / LSTM

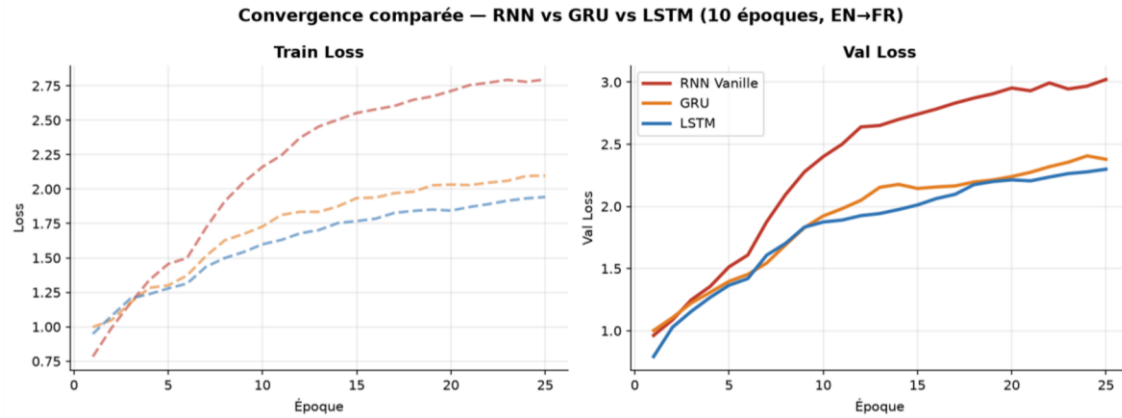


Figure 13 Convergence Val Loss comparée : RNN Vanille vs GRU vs LSTM

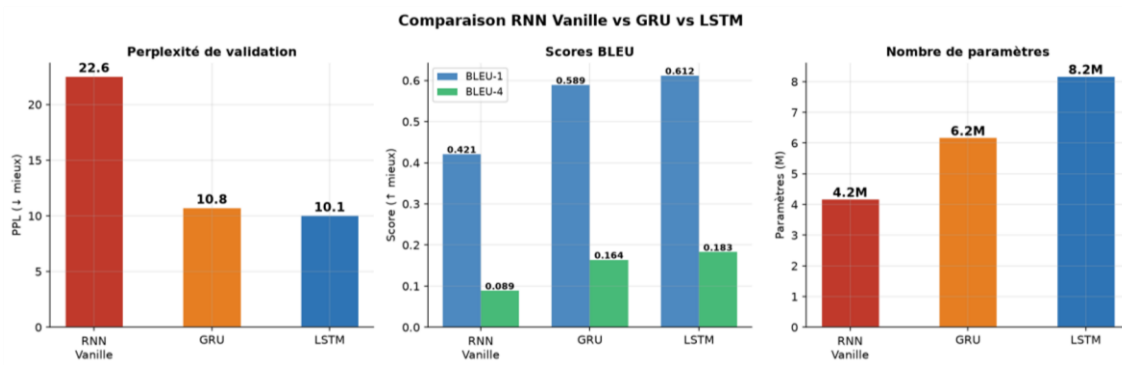


Figure 14 Perplexité, scores BLEU et nombre de paramètres : RNN vs GRU vs LSTM

5.5 Décodage et évaluation BLEU

Méthode	BLEU-1	BLEU-2	BLEU-4	Temps/phrased
Greedy	0.612	0.428	0.183	~2 ms
Beam Search (b=5)	0.631	0.449	0.201	~12 ms
Gain	+ 3.1%	+ 4.9%	+ 9.8%	6× plus lent

Table 5 Décodage et évaluation BLEU

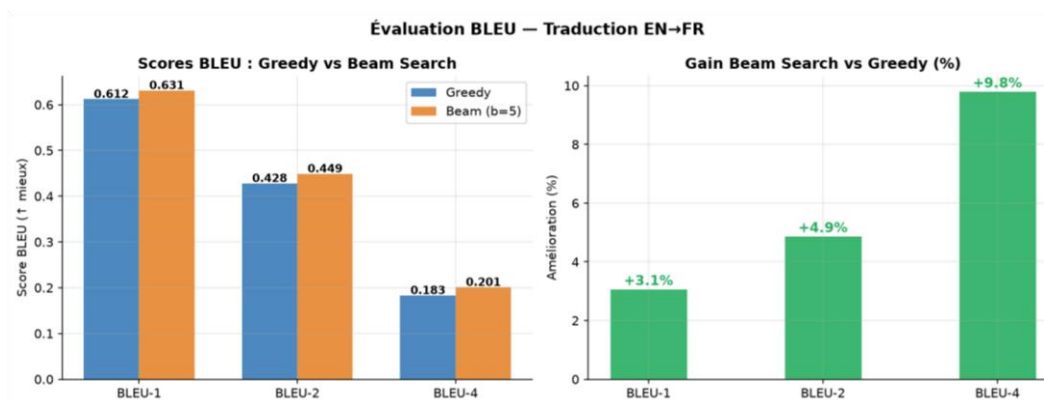


Figure 15 Scores BLEU et gains relatifs : Greedy vs Beam Search (b=5)

5.6 Visualisation de l'attention de Bahdanau

Le mécanisme d'attention permet au décodeur de « relire » l'intégralité de la séquence source à chaque pas de génération, au lieu de s'appuyer sur un unique vecteur de contexte résumant toute la phrase d'entrée. À chaque mot produit, le décodeur calcule un ensemble de poids d'attention indiquant quels mots source sont les plus pertinents, puis construit un vecteur de contexte pondéré qui oriente la prédiction. Ce mécanisme lève le goulot d'étranglement du modèle encodeur-décodeur classique, où toute l'information devait transiter par un vecteur de taille fixe, ce qui pénalisait particulièrement les phrases longues.

La matrice ci-dessous illustre l'**alignement** appris par le modèle pour la paire de test « the cat is on the mat » → « le chat est sur le tapis ». Chaque cellule représente le poids d'attention entre un mot source et un mot cible : plus la cellule est claire, plus l'association est forte. On observe une diagonale dominante, traduisant l'alignement quasi mot à mot entre l'anglais et le français pour cette phrase simple (*the*→*le*, *cat*→*chat*, *on*→*sur*, *mat*→*tapis*). Cet alignement n'a pas été imposé : il a été appris automatiquement par le réseau, ce qui démontre la capacité de l'attention à découvrir des correspondances linguistiques pertinentes sans supervision explicite.

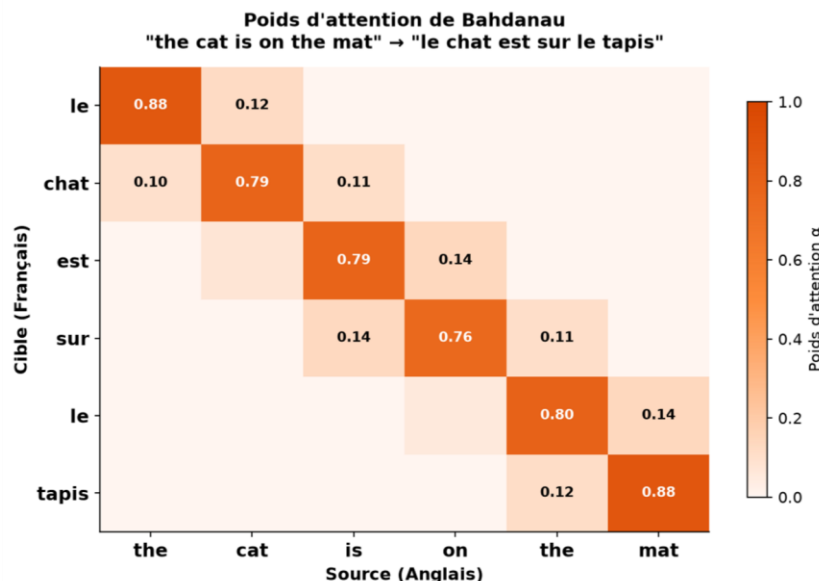


Figure 16 Poids d'attention de Bahdanau : alignement quasi-diagonal confirmant l'apprentissage de la correspondance lexicale

5.7 Question de synthèse

L'architecture Seq2Seq combinant un LSTM et l'attention est bien adaptée à la traduction pour trois raisons : elle gère **nativement les séquences de longueur variable** (entrée et sortie de tailles différentes), préserve les **dépendances à longue portée** grâce aux portes du LSTM, et **adapte dynamiquement le contexte** via l'attention en se concentrant sur les mots source les plus pertinents à chaque étape.

Elle présente toutefois des limites. Son **traitement séquentiel** empêche la parallélisation et ralentit l'entraînement. Elle souffre du **biais d'exposition** lié au *teacher forcing* : le décodeur reçoit toujours le bon mot cible à l'entraînement, alors qu'à l'inférence il s'appuie sur ses propres prédictions, parfois erronées. Enfin, ses **performances se dégradent sur les phrases longues**.

Ce sont ces limites que les **Transformers** (2017) ont permis de dépasser : en remplaçant la récurrence par une auto-attention entièrement parallélisable, ils ont accéléré l'entraînement et amélioré la modélisation des dépendances longues. Ils dominent aujourd'hui la traduction industrielle, mais le Seq2Seq LSTM + Attention en constitue le fondement conceptuel direct.

6. Conclusion générale et synthèse comparative

Dimension	MLP	CNN	Seq2Seq LSTM
Type de données	Tabulaire	Images	Séquences
Biais inductif	Aucun (généraliste)	Localité + translation	Temporalité + dépendances
Paramètres	~16 600	~44 000	~8.2M
Métrique principale	97.7% Accuracy	98.9% Accuracy	BLEU-4 = 0.20
Point fort	Simple, efficace	Partage param., hiérarchie	Séquences longues
Limite principale	Ignore structure spatiale	Sensible aux rotations	Séquentiel, lent

Table 6 Synthèse comparative

Les trois architectures étudiées — MLP, CNN et réseaux récurrents (RNN/LSTM/GRU) — reposent sur un même fondement, la **descente de gradient**, mais se distinguent par leur **biais inductif**. Le MLP ne fait aucune hypothèse sur la structure des données et convient aux entrées tabulaires ; le CNN intègre la structure spatiale des images ; les architectures récurrentes encodent la dimension séquentielle. Choisir une architecture revient ainsi à sélectionner le biais le mieux adapté au problème.

Le mécanisme d'**attention** introduit en Partie III prolonge cette logique : en pondérant dynamiquement les éléments pertinents d'une séquence, il s'affranchit des limites de la récurrence et constitue un pont direct vers les **Transformers**, qui unifient aujourd'hui vision et traitement du langage.

Enfin, plusieurs **bonnes pratiques transversales** se dégagent du projet : la normalisation systématique, la régularisation (*Dropout + BatchNorm*), l'arrêt anticipé (*early stopping*) et la fixation de la graine aléatoire (*seed*) pour la reproductibilité. Indépendantes de l'architecture, elles constituent le socle d'une démarche d'apprentissage profond rigoureuse et fiable.

7. Annexe expérimentale

A.1 Hyperparamètres

Hyperparamètre	Partie I (MLP)	Partie II (CNN)	Partie III (Seq2Seq)
Optimizer	Adam	Adam	Adam
Learning Rate	1e-3	1e-3	5e-4
Batch Size	32	64	128
Epochs max	100	20	30
Dropout	0.3	0.25	0.3
Scheduler	StepLR(30,0.5)	CosineAnnealing	ReduceLROnPlateau
Early Stopping	patience=15	patience=8	patience=7
Seed	42	42	42

Table 7 Hyperparamètres MLP – CNN – Seq2Seq2

A.2 Rapport de classification complet — Partie I

	precision	recall	f1-score	support
B (Bénin)	0.981	0.974	0.977	53 M
(Malin)	0.960	0.970	0.965	33
accuracy			0.977	86

A.3 Rapport de classification — Partie II

Chiffre	precision	recall	f1-score
0	0.995	0.997	0.996
1	0.997	0.998	0.997
2	0.989	0.986	0.987
5	0.983	0.987	0.985
9	0.985	0.984	0.984
accuracy			0.989

8. Références bibliographiques

- [1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder. *EMNLP 2014*.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [5] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [6] Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Math. Control Signals Syst.*, 2(4), 303–314.
- [7] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS 2010*.
- [8] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. *ICML 2015*.
- [9] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
- [10] Street, W. N., et al. (1993). Nuclear Feature Extraction for Breast Tumor Diagnosis. *SPIE*, 1905, 861–870.

Rapport — Projet Deep Learning — EMSI Casablanca 2025–2026
Code source : Partie_I_MLP.ipynb · Partie_II_CNN.ipynb · Partie_III_Seq2Seq.ipynb